

solver.py: 74%

132

37

0

7



```

1 import os
2 import glob
3 import sys
4 from pathlib import Path
5 from dataclasses import dataclass
6 from openpyxl import load_workbook, Workbook
7
8 import pandas as pd
9
10 @dataclass
11 class AssignmentRow:
12     faculty_id: str
13     term_label: str
14     term_code: str
15     class_id: str
16     class_label: str
17     weight: int
18
19 # Links to where data will be stored
20 BASE_DIR = Path(__file__).resolve().parent
21 DATA_DIR = BASE_DIR / ("data")
22 OUTPUT_XLSX = DATA_DIR / "solution.xlsx"
23 TENURE_CSV = DATA_DIR / "tenure.csv"
24 MAX_CLASSES_PER_FACULTY = 4 # Do not give more than N classes to any given prof
25
26 def compute_weight(tenure: int, desire: int, comfort: int) -> int:
27
28
29     # Find weight based on tenure, desire, and comfort ratings
30     weight = 0 # Default
31
32
33     # THE MEAT
34     if tenure <= 10:
35         if desire > 3 and comfort > 3:
36             weight = 4
37         elif desire < 2 and comfort < 2:
38             weight = 2
39     elif tenure > 11:
40         if desire < 2 and comfort < 2:
41             weight = 1
42         elif desire > 3 and comfort <= 3: # pylint: disable=chained-comparison
43             weight = 3
44
45     return weight
46
47 def safe_sheet_name_from_faculty(faculty_id: str) -> str:
48
49
50     # Turning facultyId into an Excel sheet name
51     # Allows each sheet inside the workbook to be an individual professor for ease of access
52
53     base = faculty_id or "Sheet"
54     if "@" in base:
55         base = base.split("@")[0]
56
57     for ch in [":", "\\", "/", "?", "*", "[", "]"]:
58         base = base.replace(ch, "_")

```

39→45

solver.py: 74%



```

61
62
63     # Making sure name stays under Excel's limits for sheets
64     return base[:31]
65
66 def load_tenure_map() -> dict[str, int]:
67     tenure_map: dict[str, int] = {}
68     if not TENURE_CSV.exists():
69         return tenure_map
70
71     with TENURE_CSV.open("r", encoding="utf-8") as f:
72         lines = [line.strip() for line in f.readlines() if line.strip()]
73
74
75     # Expect header: facultyKey,tenure
76     for line in lines[1:]:
77         parts = line.split(",")
78         if len(parts) < 2:
79             continue
80         faculty_key = parts[0].strip()
81         try:
82             tenure_val = int(parts[1].strip())
83         except ValueError:
84             continue
85         if faculty_key:
86             tenure_map[faculty_key] = tenure_val
87
88     return tenure_map
89
90 def append_to_excel(row: AssignmentRow, file_path: Path = OUTPUT_XLSX) -> bool:
91     headers = ["facultyId", "termLabel", "termCode", "classId", "classLabel", "weight"]
92     sheet_name = safe_sheet_name_from_faculty(row.faculty_id)
93
94
95     # Loads or creates workbook
96     if os.path.exists(file_path):
97         wb = load_workbook(file_path)
98     else:
99         wb = Workbook()
100         if "Sheet" in wb.sheetnames and len(wb.sheetnames) == 1:
101             wb.remove(wb["Sheet"])
102
103
104     # Sheet in said workbook
105     if sheet_name in wb.sheetnames:
106         ws = wb[sheet_name]
107     else:
108         ws = wb.create_sheet(sheet_name)
109
110
111     # Append the headers
112     if ws.max_row == 1 and all(cell.value is None for cell in ws[1]):
113         ws.delete_rows(1, 1)
114         ws.append(headers)
115
116
117     # Check for existing entries
118     existing_entries = set()
119     for xrow in ws.iter_rows(min_row=2, values_only=True):
120         if xrow and len(xrow) >= 5:
121             existing_entries.add((xrow[0], xrow[1], xrow[2], xrow[3], xrow[4]))
122

```

78→79

solver.py: 74%



```

125     wb.save(file_path)
126     return False
127
128     ws.append([row.faculty_id, row.term_label, row.term_code, row.class_id, row.class_label, row.weight])
129     wb.save(file_path)
130     return True
131
132 def process_csv_file(csv_path: Path, tenure_map: dict[str, int]) -> list[AssignmentRow]:
133     candidates = []
134     stem = csv_path.stem
135     without_prefix = stem[5:] if stem.startswith("pref_") else stem
136     parts = without_prefix.split("_")
137     faculty_key = "_".join(parts[:-1]) # Everything except last bit
138
139     df = pd.read_csv(csv_path)
140
141     # Map existing columns to solver inputs
142     # Rating -> comfort, DesireRating -> desire
143     comfort_col = "rating"
144     desire_col = "desireRating"
145
146     # Clean, ensure numbers
147     df[comfort_col] = df[comfort_col].astype(int)
148     df[desire_col] = df[desire_col].astype(int)
149
150     # Placeholder tenure
151     tenure_value = tenure_map.get(faculty_key, 10)
152     df["tenure"] = int(tenure_value)
153
154     selected_columns = [
155         "facultyId",
156         "termLabel",
157         "termCode",
158         "classId",
159         "classLabel",
160         "tenure",
161         desire_col,
162         comfort_col,
163     ]
164     for _, df_row in df[selected_columns].iterrows():
165         faculty_id = df_row["facultyId"]
166         term_label = df_row["termLabel"]
167         row_term_code = str(df_row["termCode"])
168         class_id = df_row["classId"]
169         class_label = df_row["classLabel"]
170         tenure = int(df_row["tenure"])
171         desire = int(df_row[desire_col])
172         comfort = int(df_row[comfort_col])
173
174         weight = compute_weight(tenure, desire, comfort)
175         candidates.append(AssignmentRow(
176             faculty_id=faculty_id,
177             term_label=term_label,
178             term_code=row_term_code,
179             class_id=class_id,
180             class_label=class_label,
181             weight=weight,
182         ))
183     return candidates
184
185 def collect_candidates(files: list[str], tenure_map: dict[str, int]) -> tuple[list[AssignmentRow], bool]
186     candidates = []

```



solver.py: 74%

```

189     for csv_path in files:
190         candidates.extend(process_csv_file(Path(csv_path), tenure_map))
191         if candidates:
192             wrote_any_rows = True
193
194     return candidates, wrote_any_rows
195
196 def apply_assignment(candidates: list[AssignmentRow]) -> None:
197     if not candidates:
198         return
199
200     # Sort high weight first
201     candidates.sort(key=lambda c: c.weight, reverse=True)
202
203     faculty_load: dict[str, int] = {}
204     assigned_classes: set[tuple[str, str]] = set()
205
206     for c in candidates:
207         f = c.faculty_id
208         t = c.term_code
209         cls = c.class_id
210
211         # Skip if max load
212         if faculty_load.get(f, 0) >= MAX_CLASSES_PER_FACULTY:
213             continue
214
215         if (t, cls) in assigned_classes:
216             continue
217
218         append_to_excel(
219             c,
220             OUTPUT_XLSX,
221         )
222         faculty_load[f] = faculty_load.get(f, 0) + 1
223         assigned_classes.add((t, cls))
224
225 def run_solver_for_all_prefs(term_code: str | None = None, data_dir: Path = DATA_DIR) -> Path:
226     # Scans all preference CSVs in folder 'data', computes weights, and writes solution.xlsx
227     # Each faculty gets their own sheet with only their rows
228
229     tenure_map = load_tenure_map()
230     pattern = data_dir / "pref_*.csv"
231     files = list(glob.glob(str(pattern)))
232
233     if term_code:
234         filtered_files = []
235         for f in files:
236             stem = Path(f).stem
237             parts = stem.split("_")
238             last_part = parts[-1] if parts else ""
239             if last_part == term_code:
240                 filtered_files.append(f)
241         files = filtered_files
242
243     if not files:
244         if term_code:
245             raise RuntimeError(f"No Preference CSVs found for termCode {term_code}")
246         # Linter doesn't want an else: statement here... oh well
247         raise RuntimeError(f"No preference CSVs found matching {pattern}")
248
249     # Clean workbook each time, less hassle
250     if OUTPUT_XLSX.exists():

```

197→198

239→235

244→245

solver.py: 74%



```
253 | candidates, wrote_any_rows = collect_candidates(files, tenure_map)
254 |
255 | apply_assignment(candidates)
256 |
257 | if not wrote_any_rows:                                257→258
258 |     wb = load_workbook(OUTPUT_XLSX)
259 |     if "Sheet" in wb.sheetnames and len(wb.sheetnames) == 1:
260 |         ws = wb["Sheet"]
261 |         ws["A1"] = (
262 |             f"No matching preference rows found for termCode {term_code}"
263 |             if term_code
264 |             else "No matching preference rows found"
265 |         )
266 |         wb.save(OUTPUT_XLSX)
267 |
268 | return OUTPUT_XLSX
269 |
270 | if __name__ == "__main__":                            270→271
271 |     term_code_arg = sys.argv[1] if len(sys.argv) > 1 else None
272 |     out = run_solver_for_all_prefs(term_code_arg)
273 |     if term_code_arg:
274 |         print(f"Solved for termCode {term_code_arg}. Results written to {out}")
275 |     else:
276 |         print(f"Solved for all terms. Results written to {out}")
```

« prev ^ index » next coverage.py v7.12.0, created at 2025-12-02 19:55 -0700